

Learning How Computer Networks Work and How to Troubleshoot Them

Hands-on exercises from a single cable to a diagnosed enterprise fault. One lab per video lesson, all run in a virtual lab you build in Lab 03.

How to use this guide

Each lab matches one lesson. Watch the lesson, then do the lab. Every lab has the same parts:

- **Objective:** what you will be able to show when you finish.
- **Prerequisites:** the lab state and earlier labs you need.
- **Procedure:** numbered steps. Commands are given for each platform.
- **Expected results:** what a correct run looks like. If you see something else, go back to the step where the output first changed.
- **Checkpoint:** questions to answer in your notebook before you continue.

Write every observation in the Troubleshooting Log (Appendix A) or a notebook. From Lab 04 on, write a plan before you change anything. The habit is the point of the course.

Command conventions

Commands are grouped by platform. Pick the line for your system. Parts in angle brackets are placeholders you replace.

```
win> ipconfig /all           # Windows: PowerShell or Command Prompt
mac$ ifconfig en0           # macOS: Terminal
lnx$ ip addr show           # Linux: any shell; prefix sudo where noted
vm$ ...                     # run inside a lab VM (Ubuntu Server)
sw$ ...                     # run on the GNS3 Open vSwitch console
all$ ...                    # same command on every platform
```

The reference lab

All labs run on a virtual lab. You build it in Lab 03 and reuse it for the rest of the course. Free tools are used throughout:

Role	Tool
Hypervisor	VirtualBox (Windows/Linux/Intel Mac) or UTM (Apple silicon)
Network emulator	GNS3 with Open vSwitch, for switching and VLAN labs
Router / firewall	OPNsense VM (routing, DHCP, DNS, NAT, firewall)
Lab hosts	Two or three Ubuntu Server VMs: <code>host-a</code> , <code>host-b</code> , <code>srv-1</code>
Packet capture	Wireshark (desktop), tcpdump (VMs)
Scanner	nmap
DNS query	dig (macOS/Linux), Resolve-DnsName or nslookup (Windows)
Performance	iperf3, mtr (macOS/Linux), pathping (Windows)

AUTHORIZATION RULE

Only capture, scan, or change networks you own or have written permission to test. Labs that touch your home network say so. Everything else stays inside the isolated lab.

Map Your Home Network

OBJECTIVE

List every device on your home network and draw how each one reaches the others and the internet.

PREREQUISITES

Your home network, a computer on it, and admin access to your home router. No lab is needed yet.

PROCEDURE

1. Walk the house. Write down every device that uses the network: phones, laptops, TVs, speakers, printers, cameras, consoles.
2. Mark each device as wired or wireless.
3. Find your router's address (your default gateway):

```
win> ipconfig | findstr Gateway
mac$ route -n get default | grep gateway
lnx$ ip route | grep default
```

4. Open that address in a browser and log in. Find the page that lists connected clients (often called DHCP Clients, Attached Devices, or Client List).
5. Match the router's list to your walk-through list. Note any device you can't identify.
6. Draw the map: modem → router → switch or access points → devices. Label each link as cable or Wi-Fi.

EXPECTED RESULTS

- A device list with 10–40 entries in most homes.
- A diagram in which every device has exactly one path to the router.
- Any unknown device is flagged for follow-up in Lab 25.

CHECKPOINT

1. Which device is the single point every device depends on?
2. Which devices are clients, which are servers, and are any peers?
3. Is your home network a LAN, a WAN, or both? Where is the boundary?

Watch One Message Leave Your Machine

OBJECTIVE

Send one message and find its name, its source address, its destination address, and its path.

PREREQUISITES

Wireshark installed on your computer (on macOS/Linux, allow capture permissions when the installer asks).

PROCEDURE

1. Open Wireshark and start a capture on your active interface (Wi-Fi or Ethernet).
2. In the display filter bar, type `icmp` and press Enter.
3. Send a single ping:

```
win> ping -n 1 example.com
mac$ ping -c 1 example.com
lnx$ ping -c 1 example.com
```

4. Stop the capture. Click the Echo (ping) request packet.
5. In the detail pane, expand the Internet Protocol section. Record the Source and Destination addresses.
6. Change the filter to `dns`. Find the query for `example.com` that ran just before the ping.

EXPECTED RESULTS

- The source is your machine's IP address. The destination is `example.com`'s IP address, which was learned from the DNS reply.
- The ping request is paired with an Echo reply that has the source and destination swapped.

CHECKPOINT

1. Which part of this exchange was the name, which was the address, and which was the path?
2. If the DNS query had failed, would the ping have been sent? What error would you have seen?
3. Why did your machine need two separate exchanges to send one ping?

Build the Isolated Lab

OBJECTIVE

Build the virtual lab used for the rest of the course: two Ubuntu hosts and an OPNsense router on an isolated network, with a clean snapshot you can restore.

PREREQUISITES

A computer with 16 GB RAM (8 GB is the minimum) and 60 GB free disk. Download VirtualBox or UTM, the Ubuntu Server LTS ISO, and the OPNsense DVD ISO. Install GNS3 later, before Lab 08.

PROCEDURE

1. Create an internal-only network named `labnet`. In VirtualBox, set the adapter to *Internal Network*. In UTM, use *Host Only* with a separate bridge. This network does not touch your home LAN.
2. Create the VM `rtr-1` (OPNsense, 1 vCPU, 1 GB RAM). Give it two adapters: adapter 1 on NAT (WAN), adapter 2 on `labnet` (LAN). Install it, then assign LAN = `10.10.10.1/24`.
3. Create `host-a` and `host-b` (Ubuntu Server, 1 vCPU, 1 GB RAM each), both with one adapter on `labnet`. During install, enable OpenSSH.
4. On each host, install the tools used throughout the course:

```
vm$ sudo apt update && sudo apt install -y tcpdump nmap dnsutils \
      traceroute mtr-tiny iperf3 curl net-tools ethtool netcat-openbsd
```

5. Check that each host got an address from `rtr-1`, then ping across:

```
vm$ ip -br addr
vm$ ping -c 3 <other-host-ip>
vm$ ping -c 3 10.10.10.1
```

6. Label each VM's adapters with its role in the hypervisor notes field (for example, "`rtr-1 nic2 → labnet LAN`").
7. Shut down all VMs and take a snapshot of each named `baseline`.
8. Test the reset: change a host's hostname, restore `baseline`, and confirm the change is gone.

EXPECTED RESULTS

- Both hosts have `10.10.10.x` addresses and can ping each other and `10.10.10.1`.

- Your home devices can't reach 10.10.10.0/24.
 - A baseline snapshot exists for every VM, and restoring one works.
-

CHECKPOINT

1. What exactly stops lab traffic from reaching your home LAN?
2. What does a simulator teach that this emulated lab does not, and the reverse?
3. Write your reset procedure in three lines.

Plan Before You Touch

OBJECTIVE

Write a complete diagnostic plan for a described fault without running a single command.

PREREQUISITES

A blank Troubleshooting Log (Appendix A).

THE FAULT

"Since this morning I can't get to the company wiki from my laptop. Email works. My colleague at the next desk says the wiki is fine for her. I restarted my laptop twice."

PROCEDURE

1. **Define.** Rewrite the complaint as one precise sentence: who is affected, what fails, since when.
2. **Scope.** Answer "just me or everyone?" from the facts you have. Put the fault in one place: local, path, or remote.
3. **Gather facts.** List five facts you would collect, in order, and what each would rule in or out.
4. **Hypothesize.** Write three ranked hypotheses. Give one line of reasoning for each.
5. **Test.** For the top hypothesis, write one test and the result that would confirm it or rule it out.
6. **Escalation.** Write the condition under which you would hand the fault to someone else, and what evidence you would pass along.

EXPECTED RESULTS

- The scope comes out as *local*, because a neighbor on the same path succeeds.
- Hypotheses are specific to this laptop: a cached or wrong DNS entry, a hosts-file entry, a proxy or VPN setting, a browser issue.
- Each test changes or checks one thing.

CHECKPOINT

1. What did the two restarts cost, and what did they tell you?
2. Why does "email works" narrow the fault more than it seems to?
3. Which of your hypotheses would a packet capture settle fastest?

Trace a Web Request Through the Stack

OBJECTIVE

Match each header in a real captured web request to its layer, and describe encapsulation using that request.

PREREQUISITES

Wireshark on your computer. An internet connection.

PROCEDURE

1. Start a Wireshark capture. Fetch an unencrypted page so every layer is readable:

```
win> curl.exe http://neverssl.com
mac$ curl http://neverssl.com
lnx$ curl http://neverssl.com
```

2. Stop the capture. Apply the filter `http` and select the GET packet.
3. In the detail pane, expand each section from top to bottom: Frame, Ethernet II, Internet Protocol, Transmission Control Protocol, Hypertext Transfer Protocol.
4. Draw a table with the columns OSI layer, TCP/IP layer, Wireshark section, and one key field with its value.
5. Select the matching HTTP/1.1 200 OK response. Note which fields swapped direction.

EXPECTED RESULTS

- Link: source and destination MAC addresses (the destination is your router, not the web server).
- Network: source and destination IP addresses. Transport: ports, with a destination of 80. Application: the method, host, and path.
- Each header wraps the one after it. That nesting is encapsulation.

CHECKPOINT

1. Why is the destination MAC address your router's, while the destination IP address is the server's?
2. Map these symptoms to a layer: no link light, "connection refused", "404", "host not found".
3. Which layers change at each router hop, and which stay the same end to end?

Diagnose a Damaged Cable

OBJECTIVE

Recognize what a physical-layer fault looks like from the host, using interface counters and simulated link failures.

PREREQUISITES

The Lab 03 baseline. Optional: a real patch cable and a cheap cable tester, for the physical steps.

PROCEDURE

1. On host-a, start a continuous ping to host-b and record the baseline loss and latency.
2. Pull the virtual cable. In VirtualBox, open the VM's Network settings and untick *Cable Connected*. In UTM, disconnect the adapter. Watch the ping.
3. Reconnect the cable. Now simulate a noisy line with 10% random loss plus corruption:

```
vm$ sudo tc qdisc add dev enp0s3 root netem loss 10% corrupt 2%
vm$ ping -c 50 <host-b-ip>
vm$ ip -s link show enp0s3
vm$ sudo tc qdisc del dev enp0s3 root
```

4. Physical option: test a known-good cable on the tester and record the pin-by-pin result. Then test a cable with one conductor cut or a bad crimp, and record which pair fails.

EXPECTED RESULTS

- A disconnected cable gives an immediate "Destination Host Unreachable" or timeouts, and `ip link` shows `NO-CARRIER`.
- The noisy line gives intermittent loss. The link stays up while the application misbehaves. This is the fault that looks like everything else.
- A tester shows a missing or crossed pair on the damaged cable.

CHECKPOINT

1. Why is a damaged-but-connected cable harder to diagnose than a missing one?
2. Which counters would you check on a real switch port for the same fault?
3. When is "replace the cable" the right first move?

Read Your Link: MAC, Speed, Duplex

OBJECTIVE

Find your interface's MAC address, confirm the negotiated speed and duplex, and see an Ethernet frame header.

PREREQUISITES

The Lab 03 baseline, plus your own computer.

PROCEDURE

1. Find your MAC address:

```
win> getmac /v
mac$ ifconfig en0 | grep ether
lnx$ ip link show
```

2. Check the link speed and duplex:

```
win> Get-NetAdapter | ft Name, LinkSpeed, FullDuplex
mac$ networksetup -getMedia "Ethernet"
lnx$ sudo ethtool eth0 | grep -E "Speed|Duplex|Auto"
```

3. On a lab host, capture a few frames and print the link-layer headers:

```
vm$ sudo tcpdump -e -c 5 -i enp0s3
```

4. If you have physical gear, look at the port LEDs on your computer and switch. Record what solid, blinking, off, and amber mean for that model (check the manual).

EXPECTED RESULTS

- A 48-bit MAC address. The first three bytes identify the vendor.
- Speed is 1000 Mb/s (or 100 or 2500) at full duplex, with auto-negotiation on.
- Each `tcpdump -e` line shows the source MAC > destination MAC and the EtherType.

CHECKPOINT

1. What symptoms would a duplex mismatch produce, and why doesn't the link go down?
2. Why does a frame need a MAC address when the packet inside already has an IP address?

3. Your port shows 100 Mb/s on gigabit gear. Name two likely causes.

Watch a Switch Learn

OBJECTIVE

Read a switch's MAC address table and watch it learn a newly connected device.

PREREQUISITES

GNS3 installed. Build a topology with an Open vSwitch node (sw-1) and three hosts: your Ubuntu VMs added as GNS3 VirtualBox nodes, or GNS3's built-in VPCS nodes.

PROCEDURE

1. Connect host-a and host-b to sw-1. Leave the third host unconnected. Start the nodes.
2. Open the sw-1 console and view the MAC table:

```
sw$ ovs-appctl fdb/show br0
```

3. Ping from host-a to host-b, then view the table again.
4. Connect the third host, have it ping host-a, and view the table once more. Note the port and age of the new entry.
5. Start a Wireshark capture on the host-b link, then ping from the third host to host-a. Look for the ARP broadcast that host-b also receives.
6. Stretch step: add a second switch and connect two cables between the switches to make a loop. With spanning tree off, watch the broadcast flood. Then turn it on: `ovs-vsctl set bridge br0 stp_enable=true`.

EXPECTED RESULTS

- The table starts empty or small, and gains one entry for each host that sends a frame.
- Broadcasts (ARP) reach every port. Unicast goes only to the learned port.
- A loop without spanning tree makes CPU load and traffic spike. With spanning tree on, one link is blocked.

CHECKPOINT

1. How does a switch learn a MAC address: from the source or the destination field?
2. What does a switch do with a frame for a MAC address it hasn't learned yet?
3. Why is a loop catastrophic at layer 2 but only a nuisance at layer 3?

Segment With Two VLANs

OBJECTIVE

Set up VLANs 10 and 20 on sw-1, confirm the hosts in them are isolated, and read VLAN tags on a trunk.

PREREQUISITES

The Lab 08 GNS3 topology with four hosts on sw-1.

PROCEDURE

1. Set ports 1–2 as access ports in VLAN 10 and ports 3–4 in VLAN 20:

```
sw$ ovs-vsctl set port eth1 tag=10
sw$ ovs-vsctl set port eth2 tag=10
sw$ ovs-vsctl set port eth3 tag=20
sw$ ovs-vsctl set port eth4 tag=20
```

2. Give all four hosts addresses in the same subnet (10.10.50.11–.14/24) so that only the VLAN separates them.
3. Ping within each VLAN, then ping across VLANs. Record each result.
4. Add a second switch and make the link between the switches a trunk that carries both VLANs:

```
sw$ ovs-vsctl set port eth0 trunks=10,20
```

5. Capture on the trunk link in Wireshark with the filter `vlan`. Find the 802.1Q tag and its VLAN ID.
6. Break it on purpose: remove VLAN 20 from the trunk. Observe the failure and record the symptom.

EXPECTED RESULTS

- Pings within a VLAN succeed. Pings across VLANs fail, even though the hosts share a subnet.
- Trunk frames carry an 802.1Q header with ID 10 or 20. Access-port frames are untagged.
- With VLAN 20 pruned from the trunk, only VLAN 20 hosts on the far switch go dark.

CHECKPOINT

1. What must be added for VLAN 10 and VLAN 20 hosts to talk? (Answered in Lab 14.)
2. Describe the symptom of a native-VLAN mismatch between two switches.

3. A single user "can't reach anything" after a desk move. What VLAN check comes first?

Survey Your Wireless Environment

OBJECTIVE

Measure signal strength room by room, find channel congestion, and decide whether a wireless complaint comes from the client or the access point.

PREREQUISITES

A laptop with Wi-Fi on your home network. This lab uses your real network, not the virtual lab: VMs can't see radio.

PROCEDURE

1. Read the current connection details: SSID, band, channel, signal, and PHY rate.

```
win> netsh wlan show interfaces
mac$ Option-click the Wi-Fi menu icon (or: system_profiler SPAirPortDataType)
lnx$ iw dev wlan0 link
```

2. List the nearby networks and the channels they use:

```
win> netsh wlan show networks mode=bssid
mac$ Wireless Diagnostics → Window → Scan
lnx$ nmcli -f SSID,CHAN,SIGNAL,BARS dev wifi list
```

3. Count the networks on each channel on 2.4 GHz and on 5 GHz.
4. Walk to five rooms. In each room, record signal (dBm or %), PHY rate, and a ping to the gateway (ping -c 20, or -n 20 on Windows).
5. Stand in the worst room. Connect a second device there. If both devices are poor, the problem is coverage. If only one is poor, suspect the client.

EXPECTED RESULTS

- Signal between -50 and -67 dBm is good. Below -75 dBm, expect drops.
- 2.4 GHz channels 1, 6, and 11 are usually crowded. 5 GHz is less crowded but has shorter range.
- Latency and loss rise as the signal falls, typically behind walls, floors, and appliances.

CHECKPOINT

1. Which channel would you move your AP to, and why?

2. Your signal is strong but your speed is poor. What is likely going on?
3. Why do calls drop as you walk between two access points?

Read Interface State and Counters

OBJECTIVE

Check an interface's configuration, state, and error and drop counters, and interpret rising counts.

PREREQUISITES

The Lab 03 baseline, plus your own computer.

PROCEDURE

1. List every interface and its state:

```
win> Get-NetAdapter | ft Name, Status, InterfaceDescription
mac$ networksetup -listallhardwareports; ifconfig -a
lnx$ ip -br link
```

2. Find which interface carries default traffic when both wired and wireless are up:

```
win> Get-NetIPInterface | sort InterfaceMetric
mac$ networksetup -listnetworkserviceorder
lnx$ ip route show default
```

3. Read the counters:

```
win> Get-NetAdapterStatistics
mac$ netstat -I en0 -b
lnx$ ip -s link show enp0s3
```

4. On host-a, add netem corruption (as in Lab 06) and send 200 pings. Re-read the counters on both hosts.
5. Disable, then re-enable the interface and note each state it passes through: `sudo ip link set enp0s3 down`, then `up`.
6. Record the driver and firmware version: `ethtool -i enp0s3`. On Windows, use Device Manager → Driver.

EXPECTED RESULTS

- A healthy interface shows errors and drops at or near zero. What matters is whether counts are rising over time, not their absolute value.

- The lowest metric, or the first entry in service order, wins the default route.

CHECKPOINT

1. RX errors are climbing, but only on one host. Where do you look next?
2. What is the difference between an error and a drop?
3. Why might a laptop use Wi-Fi even when a cable is plugged in?

Read a Full IP Configuration

OBJECTIVE

Identify every field in an IP configuration and judge whether it makes sense.

PREREQUISITES

Your own computer and host-a.

PROCEDURE

1. Dump the full configuration:

```
win> ipconfig /all
mac$ ifconfig en0; scutil --dns | head -20; route -n get default
lnx$ ip addr; ip route; resolvectl status
```

2. Fill in: IPv4 address, subnet mask or prefix, default gateway, DNS servers, DHCP server, lease times, MAC address, and IPv6 addresses.
3. Classify the address: public or private (10/8, 172.16/12, 192.168/16), loopback, or link-local (169.254/16).
4. Run the sanity check. Is the gateway in the same subnet as the address? Is the DNS server reachable? Is the address unique? (Run arping on Linux.)
5. On host-a, set a static address outside the subnet, then run the sanity check again:

```
vm$ sudo ip addr flush dev enp0s3
vm$ sudo ip addr add 10.10.20.50/24 dev enp0s3
vm$ ping -c 2 10.10.10.1
```

6. Restore the baseline snapshot.

EXPECTED RESULTS

- Home and lab addresses are private. The gateway shares the network portion of your address.
- The off-subnet static address can't reach the gateway: "Network is unreachable".

CHECKPOINT

1. Which bits of 10.10.10.23/24 are the network portion, and which are the host portion?
2. What does a 169.254.x.x address tell you instantly?

3. Why can your private address still browse the internet? (See Lab 17.)

Subnetting by Hand

OBJECTIVE

Work out network, broadcast, and host ranges by hand, decide which hosts can talk directly, then check your answers in the lab.

PREREQUISITES

Pencil and paper. The Lab 03 baseline for the verification step.

PROCEDURE

1. For each host below, write the network address, broadcast address, usable range, and host count:

Host	Address	Mask
A	192.168.4.20	/26 (255.255.255.192)
B	192.168.4.61	/26
C	192.168.4.70	/26
D	10.0.8.200	/21 (255.255.248.0)
E	10.0.15.9	/21
F	10.0.16.1	/21

2. Mark every pair that can talk directly with no router in between.
3. Size a subnet: what is the smallest prefix that holds 50 hosts? 500? 2?
4. Verify in the lab. Give host-a the address 10.10.10.20/26 and host-b the address 10.10.10.70/26, then ping between them. Next, change host-b to /24 only, and ping again in both directions.
5. Restore the baseline snapshot.

EXPECTED RESULTS

- A and B share 192.168.4.0/26. C is in .64/26. D and E share 10.0.8.0/21. F is in 10.0.16.0/21.
- 50 hosts → /26. 500 hosts → /23. 2 hosts → /30 (or /31 for point-to-point links).
- With mismatched masks, one side believes the other is local and the other side does not. The result is one-way failure.

CHECKPOINT

1. Explain in one sentence how a host decides whether a destination is local or remote.
2. Why must the default gateway sit inside the host's own subnet?
3. A host can reach the internet but not a printer two desks away. What mask error could explain it?

Read a Routing Table and Trace a Path

OBJECTIVE

Read a routing table, route between two lab subnets, and interpret each hop of a traceroute to a distant host.

PREREQUISITES

The Lab 03 baseline. Add a third adapter on `rtr-1` connected to a new internal network `labnet2`, with the address `10.10.20.1/24`. Move `host-b` onto `labnet2`.

PROCEDURE

1. Read the routing table on your computer and on `host-a`:

```
win> route print -4
mac$ netstat -rn -f inet
lnx$ ip route
```

2. For three destinations (a lab host, `8.8.8.8`, and `10.10.20.5`), say which route would match. Check your answer with `ip route get <dest>` on Linux or `route -n get <dest>` on macOS.
3. Ping `host-b` from `host-a`. The router forwards between the two subnets. In OPNsense, allow LAN → OPT1 traffic under Firewall → Rules.
4. Break it: delete `host-a`'s default route (`sudo ip route del default`) and ping again. Restore it.
5. Trace a path to a distant host:

```
win> tracert example.com
mac$ traceroute example.com
lnx$ traceroute example.com (or: mtr -rwc 20 example.com)
```

6. For each hop, record the address, the latency, and a guess at who owns it: your router, your ISP, a transit network, or the destination.

EXPECTED RESULTS

- Directly connected subnets appear as `proto kernel` or `link` routes. Everything else follows `default via`.
- Without a default route, you get "Network is unreachable" for any remote destination.

- * * * in a traceroute is often just a router that doesn't answer probes. It is not a failure unless every later hop is also silent.

CHECKPOINT

1. Routes exist for 10.0.0.0/8 and 10.10.0.0/16. Which one does 10.10.20.5 use, and why?
2. How can a packet reach its destination while the reply never comes back?
3. Latency jumps at hop 6 and stays high. What does that tell you, compared with a jump at hop 6 that drops again at hop 7?

Watch a DHCP Lease Happen

OBJECTIVE

Capture the full DORA exchange (Discover, Offer, Request, Acknowledge), release and renew a lease, and see what DHCP failure looks like.

PREREQUISITES

The Lab 03 baseline, with DHCP enabled on rtr-1 LAN (Services → DHCPv4).

PROCEDURE

1. On host-b, start a capture: `sudo tcpdump -ni enp0s3 -vv port 67 or port 68`.
2. On host-a, release and renew:

```
win> ipconfig /release && ipconfig /renew
mac$ sudo ipconfig set en0 DHCP
lnx$ sudo dhclient -r enp0s3 && sudo dhclient -v enp0s3
(netplan hosts: sudo networkctl renew enp0s3)
```

3. Match the four packets to the four DORA steps. In the Offer, find the offered address, mask, router, DNS servers, and lease time.
4. In OPNsense, create a reservation (static mapping) that pins host-a's MAC address to 10.10.10.50. Renew the lease and confirm the new address.
5. Break it: stop the DHCP service on rtr-1, then release and renew on host-a. Record the address you get.
6. Stretch step: turn host-b into a rogue DHCP server (`sudo apt install dnsmasq`, configured with `dhcp-range=10.10.10.200,10.10.10.210`). Renew on host-a and see which server wins. Then stop `dnsmasq`.
7. Restore the baseline snapshot.

EXPECTED RESULTS

- You see four broadcast or unicast messages on UDP ports 67/68 that carry options 1 (mask), 3 (router), 6 (DNS), and 51 (lease time).
- With no DHCP server, the host ends up with no IPv4 address or a 169.254.x.x address.
- With a rogue server present, whichever server answers first wins. Hosts get the wrong gateway or DNS server.

CHECKPOINT

1. Why is the Discover a broadcast?
2. What happens to existing clients when the DHCP server goes down? When does it start to matter?
3. How would you find a rogue DHCP server on a real network?

Query DNS by Hand

OBJECTIVE

Query several record types, follow a lookup from the root down, and diagnose a name that won't resolve.

PREREQUISITES

The Lab 03 baseline with internet access through `rtr-1`, plus your own computer.

PROCEDURE

1. Query A, AAAA, MX, CNAME, TXT, and NS records for a domain you know:

```
win> Resolve-DnsName example.com -Type MX
mac$ dig example.com MX +short
lnx$ dig example.com MX +short
```

2. Follow the hierarchy: `dig +trace www.example.com` (macOS/Linux). Name the root server, the TLD server, and the authoritative server.
3. Query the same name twice and compare the TTLs. Then query a specific resolver directly: `dig @1.1.1.1 example.com`.
4. Find your resolver and its lookup order:

```
win> Get-DnsClientServerAddress; type C:\Windows\System32\drivers\etc\hosts
mac$ scutil --dns; cat /etc/hosts
lnx$ resolvectl status; cat /etc/hosts
```

5. Break it: on host-a, add `10.10.10.99 example.com` to `/etc/hosts`. Compare `curl example.com` with `dig example.com`. Remove the line.
6. Break it again: point host-a's DNS at an address that doesn't exist (`10.10.10.253`). Try ping `example.com`, then ping its IP address directly.
7. Clear the cache: `ipconfig /flushdns` (Windows), `sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder` (macOS), `resolvectl flush-caches` (Linux).

EXPECTED RESULTS

- The TTL counts down on repeated queries. A cached answer comes back noticeably faster.
- The hosts-file entry overrides DNS for applications but not for `dig`, which queries DNS directly.

- With a dead resolver, pinging by name fails and pinging by IP address works. That split confirms DNS in seconds.

CHECKPOINT

1. A DNS record was changed an hour ago. Half your users see the new address. Why?
2. What is the difference between SERVFAIL, NXDOMAIN, and a timeout?
3. Why can nslookup or dig succeed while the browser still fails?

NAT and a Port Forward

OBJECTIVE

Confirm your public address, read NAT translation state, publish one lab service with a port forward, and spot double NAT.

PREREQUISITES

The Lab 03 baseline. Change `rtr-1`'s WAN adapter from NAT to *Host-only* so your computer can act as the "internet" side.

PROCEDURE

1. From your computer, compare your local address with your public one: run `curl ifconfig.me` on every platform (`curl.exe` on Windows).
2. Look at the WAN address on your home router's status page. If it is private (`10.x`, `192.168.x`, `100.64-127.x`), you are behind double NAT or carrier-grade NAT.
3. In the lab, open two connections from `host-a` to the `host-only` side. View them in OPNsense under Firewall → Diagnostics → States. Note the source port rewrites.
4. On `host-b`, start a web server: `python3 -m http.server 8080`.
5. In OPNsense, forward WAN TCP port 80 to `host-b` port 8080 (Firewall → NAT → Port Forward). From your computer, browse to `rtr-1`'s WAN address.
6. Capture on both of `rtr-1`'s sides (Interfaces → Diagnostics → Packet Capture) and compare the addresses before and after translation.

EXPECTED RESULTS

- Outbound flows appear as the WAN address with rewritten source ports. Many private hosts share one public address.
- The port forward works from the WAN side. Without the forward, the connection times out.

CHECKPOINT

1. How does return traffic find the right inside host?
2. Why does a port forward fail under double NAT, and what are the two fixes?
3. Why do some games and video calls struggle behind NAT?

Connections, States, and Ports

OBJECTIVE

List listening ports and active connections, tie each one to a process and protocol, and see what a blocked port and a closed port each look like.

PREREQUISITES

The Lab 03 baseline, plus your own computer.

PROCEDURE

1. List the listening sockets and established connections, with their processes:

```
win> Get-NetTCPConnection | sort State | ft -a; netstat -ano
mac$ sudo lsof -iTCP -sTCP:LISTEN -n -P; netstat -an -p tcp
lnx$ sudo ss -tulpn; ss -tan
```

2. For five entries, record the local port, remote port, protocol, state, and process. Name the service implied by each well-known port.
3. On host-b, listen on TCP 9000 (`nc -l 9000`) and on UDP 9001 (`nc -u1 9001`). From host-a, connect to each with `nc` and exchange a line of text.
4. Capture the TCP session and find the SYN, SYN-ACK, and ACK. Close it and find the FIN or RST packets.
5. Test a closed port (`nc -vz <host-b> 9999`), then block port 9000 with a firewall drop rule (`sudo iptables -A INPUT -p tcp --dport 9000 -j DROP`) and test again. Compare the two results.

EXPECTED RESULTS

- A closed port answers immediately with RST: "Connection refused".
- A filtered (dropped) port stays silent: a timeout after several SYN retries.
- UDP has no handshake and no connection state, so `nc` can't tell you whether anything received the data.

CHECKPOINT

1. What does "refused" prove that "timed out" does not?
2. Many connections are stuck in SYN_SENT. Where is the problem?
3. Why does voice use UDP even though packets may be lost?

A Raw Web Request by Hand

OBJECTIVE

Type an HTTP request by hand, then read the full response: status line, headers, and body.

PREREQUISITES

The Lab 03 baseline with `python3 -m http.server 8080` running on host-b.

PROCEDURE

1. From host-a, open a raw TCP connection and type the request. End it with a blank line:

```
vm$ nc <host-b-ip> 8080
GET / HTTP/1.1
Host: host-b
```

2. Read the status line and every header. Request a file that doesn't exist and record the status code you get back.
3. Do the same against a public site with verbose curl:

```
win> curl.exe -v http://example.com
mac$ curl -v http://example.com
lnx$ curl -v http://example.com
```

4. Request `http://github.com` with `curl -I` and follow the redirect chain with `-L`.
5. Check the clock on each machine (`w32tm /query /status` on Windows, `ntp -t 1 time.apple.com` on macOS, `timedatectl` on Linux) and compare it with the `Date:` header.

EXPECTED RESULTS

- 200 OK for the root, 404 for the missing file, 301/308 for the HTTP → HTTPS redirect.
- The service can answer correctly on its port and still return an error status. That is an application-layer fault, not a network fault.

CHECKPOINT

1. What is the difference in meaning between a 4xx code and a 5xx code?
2. Why is the `Host:` header required?

3. A login fails only on one server whose clock is 10 minutes off. Explain why.

Inspect a Certificate Chain

OBJECTIVE

Read a site's certificate chain, then trigger and interpret four common TLS errors.

PREREQUISITES

Your own computer with OpenSSL. It comes with macOS and Linux. On Windows, use the copy bundled with Git for Windows, or run these steps on a lab VM.

PROCEDURE

1. Open a site in a browser, click the padlock, and view the certificate. Record the subject, issuer, validity dates, and SAN names.
2. Read the same chain from the command line:

```
all$ openssl s_client -connect example.com:443 -servername example.com -  
showcerts </dev/null  
all$ ... | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
```

3. Trigger each error on purpose, using the public test hosts at badssl.com:

```
all$ curl -v https://expired.badssl.com  
all$ curl -v https://wrong.host.badssl.com  
all$ curl -v https://untrusted-root.badssl.com  
all$ curl -v --tls-max 1.0 https://example.com
```

4. For each one, write the exact error text and its meaning in plain words.
5. Hash versus encryption: run `echo hello | sha256sum` (on macOS, `shasum -a 256`) twice, then `echo hello | base64`. Say which output can be reversed.

EXPECTED RESULTS

- The chain runs leaf → intermediate → root. The root lives in your OS trust store and is not sent by the server.
- The four errors are: certificate has expired; hostname doesn't match; unable to get local issuer; protocol version or handshake failure.

CHECKPOINT

1. What does the padlock guarantee, and what does it not?
2. Only one laptop gets "certificate not yet valid". Where do you look?
3. Why does TLS use both asymmetric and symmetric encryption?

Capture and Narrate a Web Session

OBJECTIVE

Capture a full web session, filter it down to a single conversation, and narrate the DNS lookup, TCP handshake, TLS setup, request, response, and teardown.

PREREQUISITES

Wireshark on your computer. Capture only your own traffic.

PROCEDURE

1. Close any chatty apps. Start a capture, run `curl -v https://example.com` (`curl.exe` on Windows), then stop the capture.
2. Filter in steps:

```
dns.qry.name contains "example"  
ip.addr == <server-ip>  
tcp.stream eq <n>           (right-click → Follow → TCP Stream)
```

3. Label every packet in the stream: SYN, SYN-ACK, ACK, ClientHello, ServerHello and Certificate, application data, FIN.
4. Open Statistics → Conversations and Statistics → Flow Graph. Compare the flow graph with your labels.
5. Find the failure signs: use the filters `tcp.analysis.retransmission`, `tcp.flags.reset==1`, and `dns.flags.rcode != 0`. Then add netem loss on a lab host (Lab 06), capture there with `tcpdump -w`, and open the file in Wireshark to see retransmissions.

EXPECTED RESULTS

- A clean session runs DNS query and response → 3-way handshake → TLS hello exchange → encrypted data → FIN/ACK.
- Where the conversation stops tells you the layer: no SYN-ACK points to the path or a firewall, a TLS alert points to certificates or versions, a response with an error code points to the application.

CHECKPOINT

1. You see repeated SYNs with no reply. Name three possible causes.

2. Why can't you read the HTTP headers in this capture? What would you need to read them?
3. When is capturing on a network you don't own a problem, even for diagnosis?

Measure Latency, Loss, and Throughput

OBJECTIVE

Measure each performance metric separately and name which one limits a slow session.

PREREQUISITES

The Lab 03 baseline, with iperf3 on both hosts.

PROCEDURE

1. Measure baseline throughput. Run `iperf3 -s` on host-b, then run `iperf3 -c <host-b> -t 10` on host-a.
2. Measure latency and jitter to three destinations: the gateway, a regional site, and a distant site.

```
win> pathping -q 50 <dest>
mac$ mtr -rwc 50 <dest>          (brew install mtr)
lnx$ mtr -rwc 50 <dest>
```

3. On host-a, change one variable at a time and rerun iperf3 after each change. Record the results in a table:

```
vm$ sudo tc qdisc add dev enp0s3 root netem delay 80ms
vm$ sudo tc qdisc change dev enp0s3 root netem loss 1%
vm$ sudo tc qdisc change dev enp0s3 root tbf rate 10mbit burst 32kbit
latency 400ms
vm$ sudo tc qdisc del dev enp0s3 root
```

4. Bufferbloat test: while iperf3 saturates the 10 Mbit limit, ping the other host. Compare the latency with the idle latency.
5. Run a public speed test on your computer and write down which server it picked and why the result may not match what you experience.

EXPECTED RESULTS

- Added delay lowers single-stream TCP throughput only a little. 1% loss lowers it sharply.
- Under load, latency rises by tens to hundreds of milliseconds. That rise is bufferbloat.

CHECKPOINT

1. Define bandwidth, throughput, latency, and jitter in one line each.

2. Video calls stutter but downloads are fast. Which metric is to blame?
3. Why does a small amount of loss hurt TCP so much?

Design a Small-Office Network

OBJECTIVE

Design a complete network on paper: requirements, topology, addressing plan, segmentation, redundancy, and documentation.

THE BRIEF

A 25-person office. Staff laptops, 6 VoIP phones, 2 printers, a file server, 10 IP cameras, and guest Wi-Fi for visitors. It must survive one switch failure and grow to 60 staff without a redesign.

PROCEDURE

1. Write a traffic matrix: which groups must reach which other groups, and which must never reach each other.
2. Choose your segments and a VLAN ID for each (for example, staff, voice, servers, cameras, guest, management).
3. Carve an addressing plan out of 10.20.0.0/16. Size each subnet for triple the current count. Put the gateway on .1 in every subnet.
4. Draw the physical topology (gateway, core switch, access switches, APs) and the logical topology (VLANs and routing) as separate diagrams.
5. Remove single points of failure: add a second uplink per access switch and set a spanning-tree root. Note what still has no redundancy.
6. Write the documentation pack: the addressing table, VLAN table, port map, and firewall intent (as rules in plain words).
7. Optional: build the design in GNS3 with two access switches and OPNsense, using the method from Labs 08 and 09.

EXPECTED RESULTS

- At least four segments. Guest and cameras can't initiate traffic to staff or servers.
- Subnets are aligned to their boundaries and don't overlap. The plan leaves room for new segments.

CHECKPOINT

1. What chronic problems would a flat /24 for everything cause in two years?
2. Which single failure in your design would still take down the office?

3. How does your documentation make Lab 28-style troubleshooting faster?

Configure a Home Router End to End

OBJECTIVE

Set up a gateway properly: a sensible address plan, a DHCP scope with reservations, a guest network, and current firmware.

PREREQUISITES

Use `rtt-1` (OPNsense) as a stand-in for a home gateway. Optionally repeat the lab on your real router, after backing up its configuration.

PROCEDURE

1. Back up the current configuration (System → Configuration → Backups). On a real router, look for Save or Backup under Administration.
2. Write the plan first: LAN `192.168.50.0/24`, DHCP pool `.100–.199`, reservations below `.100`, guest network `192.168.60.0/24`.
3. Apply the LAN address and DHCP scope. Reserve addresses for a printer and a NAS.
4. Create the guest segment: an OPT interface (or VLAN), its own DHCP scope, and firewall rules that allow internet access and block RFC 1918 destinations.
5. Change the default admin password. Turn off WAN-side admin access and UPnP (unless it is needed).
6. Check for firmware updates and apply them. Record the version before and after.
7. From a guest host, try to reach the LAN, then the internet. Record both results.

EXPECTED RESULTS

- Guest reaches the internet, and guest access to the LAN is blocked.
- Reserved devices keep the same address after renewing their lease.

CHECKPOINT

1. List the separate devices hiding inside an all-in-one home gateway.
2. When is a mesh system the right fix, and when is an extender? When is a cable?
3. The reboot fixed it. What should you check so it doesn't happen again?

Audit Firewall, Wireless, and Management

OBJECTIVE

Review firewall rules against default-deny, check for exposed management services, and harden wireless settings.

PREREQUISITES

The Lab 24 configuration. Scan only the lab, and your own router from inside your network.

PROCEDURE

1. Export or list every firewall rule. For each rule, write its purpose in plain words. Flag any rule that is *any* → *any* or has no purpose you can state.
2. Scan the gateway from the LAN side and from the WAN side:

```
all$ nmap -sT -p- --open 10.10.10.1
all$ nmap -sU --top-ports 50 10.10.10.1
all$ nmap -sV -p 22,80,443,8080 <rtr-1-wan-ip>
```

3. For every open port, record the service and say whether it should be reachable from that side.
4. Wireless (on your real AP): confirm WPA3 or WPA2-AES, turn off WPS, drop TKIP and WEP, and set a strong passphrase. Isolate IoT devices on the guest or IoT network.
5. Look for signs of misuse: compare the DHCP client list with Lab 01, review the firewall logs for repeated blocked inbound attempts, and check for unknown port forwards.

EXPECTED RESULTS

- The WAN scan shows only the Lab 17 port forward, or nothing at all. Management services answer on the LAN only.
- The rule list ends with a default deny, and every allow rule has a stated purpose.

CHECKPOINT

1. What is the difference between a router, a firewall, and a gateway?
2. What does a VPN protect, and what does it not?
3. You find an unknown device with an open telnet port. What are your first three moves?

Map Service Dependencies

OBJECTIVE

Map the network services your environment depends on, find the single points of failure, and see a cascade happen.

PREREQUISITES

The Lab 03 baseline, plus your notes from Labs 15 and 16.

PROCEDURE

1. List every service your home or lab uses: DHCP, DNS, NTP, file shares, printing, authentication, and monitoring. For each one, note which host provides it.
2. Draw a dependency graph for one application, such as a file share, following it down to the network services it needs.
3. Circle every node that has only one provider.
4. Watch a cascade: with DNS on `rtr-1`, stop Unbound (Services → Unbound DNS). From host-a, try `apt update`, `curl`, and `ssh host-b` by name. Record what fails and the error each one gives. Restart Unbound.
5. Check time synchronization on every host (`timedatectl`, `w32tm /query /status`, `sntp`). Point the lab hosts at `rtr-1` as their NTP server.
6. Turn on basic monitoring. In OPNsense, go to Reporting → Health. Or send syslog from `rtr-1` to host-b with `rsyslog`.

EXPECTED RESULTS

- A DNS outage looks like many unrelated application failures, while everything addressed by IP keeps working.
- Most small networks have one gateway providing DHCP, DNS, and NTP: a single point of failure for all three.

CHECKPOINT

1. Why does single sign-on fail when time drifts?
2. Which single point of failure would you remove first, and how?
3. What would monitoring have told you before users noticed the outage?

Find the Provider Handoff

OBJECTIVE

Identify your connection type, find where the provider's responsibility ends, and sort faults into yours and theirs.

PREREQUISITES

Your home connection and access to the modem or ONT status page (often at 192.168.100.1 for cable modems).

PROCEDURE

1. Identify the technology from the device and the bill: cable, DSL, fiber (ONT), fixed wireless, cellular, or satellite.
2. Photograph the handoff point: the wall jack, the ONT, or the demarcation box. Label what the provider owns.
3. From the modem or ONT status page, record the sync rate, signal levels, and uptime.
4. Run a traceroute and mark the first hop outside your network, then the first hop inside the provider (Lab 14).
5. Sort the evidence into a table: faults visible before the handoff (yours) and after it (theirs).
6. Write the "before you call" packet: the time the fault started, whether the modem and ONT lights are normal, results with a computer cabled directly to the modem, the traceroute, and the loss measured to the first ISP hop.

EXPECTED RESULTS

- A clear line: your router and everything behind it are yours, and the modem or ONT and beyond are usually the provider's.
- A one-page packet that lets support skip the script.

CHECKPOINT

1. Loss starts at hop 1 (your router). Whose problem is it?
2. Loss starts at the first ISP hop, and a direct-cabled test shows the same loss. What is your next step?
3. How would you add a second, failover connection, and what would it protect against?

Diagnose a Multi-Layer Fault

OBJECTIVE

Apply the full framework to a fault you didn't cause, write down each step of your reasoning, and reach the root cause.

PREREQUISITES

The Lab 14 two-subnet topology. Set up the fault injector below so that you don't know which faults are active.

FAULT INJECTOR

Save this on host-a as `~/break.sh` and run it with `sudo bash ~/break.sh`. It applies two random faults and hides which ones.

```
#!/bin/bash
IF=enp0s3
F=(
  "tc qdisc add dev $IF root netem loss 25%"
  "ip route del default"
  "sed -i 's/^#\?DNS=.*?DNS=10.10.10.253/' /etc/systemd/resolved.conf; systemctl restart systemd-resolved"
  "ip addr add 10.10.10.1/24 dev $IF"
  "iptables -A OUTPUT -p tcp --dport 443 -j DROP"
  "ip link set $IF mtu 576"
)
for i in $(shuf -i 0-5 -n 2); do eval "${F[$i]}"; done
echo "Faults applied. Restore baseline when done."
```

PROCEDURE

1. Symptom: "host-a can't load `https://example.com`." Open a fresh Troubleshooting Log.
2. Pick an approach: bottom-up, top-down, or divide-and-conquer. Write down why you chose it.
3. Split the fault location: is it host-a, the path, or the far end? Test from host-b to split it.
4. Walk the layers. For each check, record the command, the result, and what it rules in or out.
5. When you find the first fault, fix only that. Test again. The symptom may change instead of going away.
6. Continue until the original symptom is gone. Name both root causes.
7. Restore the baseline snapshot and run the injector again for a second attempt.

EXPECTED RESULTS

- A log that another person could follow to reach the same conclusion.
- Two faults found, each tied to a specific layer and to the evidence that proved it.

CHECKPOINT

1. Which single test narrowed the fault most? Why?
2. How did the first fault hide the second?
3. Add three lines to your personal checklist based on this run.

The Toolkit, Then the Sequence

OBJECTIVE

Record what normal output looks like for each tool, then use a fixed sequence of tools to localize a staged fault.

PREREQUISITES

The Lab 03 baseline and the Lab 28 fault injector.

PROCEDURE

1. With the lab healthy, run each tool against host-b and example.com. Save the output as your known-good reference.

Question	Windows	macOS	Linux
Is my config sane?	<code>ipconfig /all</code>	<code>ifconfig; netstat -rn</code>	<code>ip a; ip r</code>
Can I reach it?	<code>ping</code>	<code>ping</code>	<code>ping</code>
Where does the path die?	<code>tracert, pathping</code>	<code>traceroute, mtr</code>	<code>traceroute, mtr</code>
Does the name resolve?	<code>Resolve-DnsName</code>	<code>dig</code>	<code>dig</code>
Is the port open?	<code>Test-NetConnection -Port</code>	<code>nc -vz</code>	<code>nc -vz</code>
What's listening?	<code>netstat -ano</code>	<code>lsof -i -P</code>	<code>ss -tulpn</code>
What's on the wire?	<code>Wireshark, pktmon</code>	<code>tcpdump</code>	<code>tcpdump</code>
What's on the network?	<code>nmap -sn</code>	<code>nmap -sn</code>	<code>nmap -sn</code>

2. Write your fixed sequence: config → gateway ping → far-IP ping → name lookup → port test → traceroute → capture.
3. Run the injector and work the sequence. Stop at the first step whose output differs from your known-good reference.
4. Repeat three times with fresh restores. Time each run.

EXPECTED RESULTS

- Each fault shows up at a predictable step. For example, the dropped default route fails at the far-IP ping, and a dead DNS server fails at the name lookup.
- Your time to localize a fault drops with each repetition.

CHECKPOINT

1. What do a ping timeout, "Destination unreachable", and "TTL expired" each mean?
2. Which staged fault did your sequence find the slowest, and why?
3. When is nmap the right tool, and when is it inappropriate?

Work the Connectivity Flowchart

OBJECTIVE

Turn "no internet" into a precise fault by working bottom-up, and do it for faults staged at different layers.

THE FLOWCHART

1. Link up? → no: check the cable, the adapter, whether the interface is enabled, the driver.
2. Valid address? → 169.254 or none: DHCP. Duplicate: ARP conflict. Wrong subnet: config.
3. Gateway pings? → no: VLAN, mask, or gateway address.
4. Far IP pings (8.8.8.8)? → no: routing, NAT, or upstream.
5. Name resolves? → no: resolver, hosts file, DNS server.
6. Service port answers? → no: firewall, or the far end is down.
7. Application works? → no: application layer, TLS, authentication.

PROCEDURE

1. Stage these faults one at a time on host-a, restoring the baseline between them: disconnect the virtual cable; stop DHCP on `rtr-1`; give host-b the same IP as host-a; set a wrong gateway; set a dead DNS server; stop the web server on host-b.
2. For each fault, start at step 1 and record the step number where it fails.
3. Make it intermittent: add `netem loss 15%`. Run ping for 5 minutes with timestamps (`ping -D` on Linux) while capturing. Find the gaps.

EXPECTED RESULTS

- Each fault stops at a different step: link, address, address (duplicate), gateway, name, service.
- The intermittent fault needs long observation. A single test passes and proves nothing.

CHECKPOINT

1. What does a duplicate address look like from each of the two hosts?
2. Everything passes up to step 6. Whose problem is it?
3. How long would you monitor an intermittent fault before drawing a conclusion?

Slow, and Sometimes Gone

OBJECTIVE

Take a staged slowdown and a staged intermittent drop, and isolate each to a specific cause and layer.

PREREQUISITES

The Lab 14 topology, with iperf3 and your Lab 22 baseline measurements.

PROCEDURE

1. **Scenario A, "slow for one user"**. On host-a, apply `tc qdisc add dev enp0s3 root tbf rate 5mbit burst 16kbit latency 300ms`. Measure with iperf3 from both hosts and compare. Check the interface speed and duplex (ethtool).
2. **Scenario B, "slow for everyone"**. On host-b, run `iperf3 -c <rtr-1> -t 300 -P 8` in the background to saturate the link. Find the top talker in OPNsense under Reporting → Insight or Diagnostics → pfTop.
3. **Scenario C, "calls drop every few minutes"**. On rtr-1's LAN side, script a link flap: a loop that disconnects the cable for 3 seconds every 90 seconds, using `VBoxManage controlvm rtr-1 setlinkstate2 off` and then on. Run a timestamped ping and a capture, and look for resets and gaps.
4. For each scenario, write the metric that moved, where it moved, the cause, and the layer.

EXPECTED RESULTS

- A: throughput is capped on one host only, while latency looks normal when the link is idle.
- B: every host sees high latency and low throughput, and one flow dominates.
- C: regular gaps with RSTs or retransmission bursts, and link-down messages in the logs.

CHECKPOINT

1. Which real-world faults could produce Scenario C? Name at least three layers.
2. How do you tell a slow network from a slow server?
3. What would you change on the network to protect voice during Scenario B?

Assess Yourself and Plan 90 Days

OBJECTIVE

Rate your skills across the course and commit to a specialization and a 90-day plan.

PROCEDURE

1. Rate yourself from 1 to 4 on each area (1 = need notes, 4 = could teach it): physical and link, switching and VLANs, wireless, addressing and subnetting, routing, DHCP and DNS, NAT, transport, TLS, capture, performance, design, security, and diagnosis.
2. For each area you rated 1 or 2, redo the matching lab without looking at the guide.
3. Choose one path: routing and switching, wireless, security, cloud networking, or automation. Write one sentence on why.
4. Plan how the lab grows: which VMs, simulators, or spare hardware you will add for that path.
5. Write a 90-day plan: weeks 1–4 consolidate the weak areas, weeks 5–8 go deeper on your path, weeks 9–12 finish a project (for example, automating the Lab 09 VLAN build with a script).
6. Pick one certification to aim for, or decide deliberately not to, and write down why.

CHECKPOINT

1. Which three ideas from this course will still be true in ten years?
2. Where will you practice, and what are you authorized to touch there?
3. What will you have built by day 90?

Troubleshooting Log

Copy this page for each fault. Fill in the header before you touch anything.

Date / time started	
Reported symptom (verbatim)	
Precise problem statement	
Scope: me / some / everyone	
Location: local / path / remote	
Reproducible? How?	
Approach and why	

#	Layer	Hypothesis	Test / command (one change)	Observed	Rules in / out
1					
2					
3					
4					
5					
6					
7					
8					

Root cause and layer	
Fix applied	
Verified by	
Prevent recurrence	